

Mini Project - Mini-RAG

Ask Your Documents

Mini Project 2 · Week 9 deliverable · ~5-6 hours over 2-3 sittings. A standalone end-to-end RAG build that mirrors the capstone's core loop at small scale, on a corpus that is NOT the capstone's.

What you're building

A small Python script that lets a user ask questions about a collection of Sherlock Holmes stories. The script:

1. Loads 5 short stories from **corpus/**
2. Chunks each story into manageable pieces
3. Embeds the chunks using OpenAI's **text-embedding-3-small**
4. Stores them in a Qdrant collection
5. Takes a question, retrieves the most relevant chunks, and answers using **gpt-4o-mini** — citing the source story and section

You'll then validate your pipeline against **2 predefined questions** and extend it with **3 questions of your own**. Submit the working script, your 5 Q&A pairs, and a short reflection.

Why this exists

You've spent W6-W9 building the capstone's RAG pipeline in pieces: ingestion in W7, structure-aware chunking in W8, hybrid+rerank in W9. The capstone is a long-running build with accumulated scaffolding, ADRs, golden sets, verifiers. By now, you've forgotten what it felt like to start from zero.

MP2 is *starting from zero* — on a small, fun corpus that has nothing to do with HR policy or VPN setup. The point isn't to learn new concepts. It's to prove to yourself (and your future portfolio reviewers) that you can compose W6-W8 from a blank file without leaning on the capstone repo.

You'll also have something concrete to point at when someone asks "*can you build a basic RAG system?*" — a question that comes up forever in this work.

The corpus

5 Sherlock Holmes stories, summarised for length (~3-5 KB each):

File	Story
01_red_headed_league.txt	The Red-Headed League
02_speckled_band.txt	The Adventure of the Speckled Band
03_blue_carbuncle.txt	The Adventure of the Blue Carbuncle

File	Story
04_engineers_thumb.txt	The Adventure of the Engineer's Thumb
05_scandal_in_bohemia.txt	A Scandal in Bohemia

The original texts are public domain (published 1892). The summaries shipped with this project are written in our own words, preserving the plots, named characters, and locations — but they're not Conan Doyle's prose. That distinction matters legally and pedagogically.

Want to use your own corpus? Drop your **.txt** files into **corpus/** (replacing or alongside the Sherlock ones), write 5 new Q&A pairs against your corpus, and submit that. The pipeline doesn't care what the corpus is. This makes MP2 a true portfolio piece. Just be honest in the reflection about what you used.

What you need to do

Step 1 — Get the environment ready (15 min)

```
cd learner/
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# Edit .env with your OPENAI_API_KEY and QDRANT_URL
source .env
```

Test that the credentials work:

```
python -c "from openai import OpenAI; print(OpenAI().models.list().data[0].id)"
python -c "from qdrant_client import QdrantClient; import os;
print(QdrantClient(url=os.environ['QDRANT_URL'],
api_key=os.environ.get('QDRANT_API_KEY')).get_collections())"
```

Both should run without error.

Step 2 — Fill in mp2_rag_starter.py (2-3 hours)

Rename it to mp2_rag.py and fill in every TODO. There are 7 functions to implement:

1. **load_corpus** — read every .txt file in corpus/
2. **chunk_document** — split into pieces (~500 chars each)
3. **embed_texts** — batch-embed via OpenAI
4. **setup_collection** — create the Qdrant collection
5. **ingest_chunks** — embed + upsert
6. **retrieve** — embed query, search Qdrant, return top-k
7. **answer** — assemble context, call LLM, return result

Once they're filled in:

```
python mp2_rag.py ingest # builds the collection (run once)
```

```
python mp2_rag.py ask          # interactive Q&A — try a few questions
```

Step 3 — Validate against the 2 predefined questions (15 min)

```
python mp2_rag.py validate
```

This runs against `data/predefined_questions.jsonl`. Both should produce answers that cite the expected source. If they don't, debug before moving on: chances are your chunking is off (chunks too big, too small, or missing section context).

Step 4 — Author your 3 questions (45 min)

Open `data/learner_questions.jsonl` and replace the 3 templates with your own questions. Vary them:

- One easy (a named character or a single fact)
- One medium (something specific, like the date of an event, or who delivered a message)
- One harder (multi-hop — needs info from two chunks; or a detail buried in the middle)

Spread them across different stories so retrieval has to discriminate.

Then run `python mp2_rag.py validate` again. All 5 should pass the source-match check, or at least come close.

Step 5 — Write your reflection (45 min)

Create `mp2_reflection.md` with these four sections:

MP2 Reflection

What worked

(One specific thing that worked — a chunking choice, a prompt detail, etc.)

What didn't work

(One specific failure mode you hit. Did chunks get too small? Did retrieval pull the wrong section? Did the LLM hallucinate?)

What I'd change

(One thing you'd do differently with another 5 hours.)

One surprise

(Anything from the build that genuinely surprised you. Could be technical — "text-embedding-3-small was way better than I expected on rare names" — or pedagogical — "I didn't realise how much hybrid retrieval was helping until I tried dense-only here.")

Be specific. "What didn't work: my chunks were too small and answers were truncated" beats "the chunking was tricky."

Step 6 (Optional STRETCH) — Try W9 hybrid+rerank (45-60 min)

If you finish Steps 1-5 with time to spare, uncomment the Stretch deps in requirements.txt, install them, and look at the `_hybrid_rerank_retrieve` function in the reference solution. Adapt or rewrite it for your own script, then add a section to your reflection: *did hybrid+rerank help on this corpus?* The answer might surprise you (narrative prose is very different from enterprise policy).

What you submit

File	What
mp2_rag.py	Your working pipeline
data/learner_questions.jsonl	Your 3 questions (replace the templates)
mp2_reflection.md	Your 4-section reflection
Output of <code>python mp2_rag.py validate</code> pasted into reflection or a separate <code>mp2_validation.txt</code>	

Push to a personal GitHub repo (separate from the capstone). Submit the repo URL.

Help

- API keys not working? Make sure `.env` uses `export VAR=...` (not `VAR=...`) and you ran `source .env`. The `export` keyword matters.
- Qdrant connection failing? Free-tier clusters can take ~60 seconds to wake from sleep. Try again.
- Pipeline runs but answers are weird? Inspect your chunks. Print them out. The most common Mini-RAG failure is bad chunking (boundaries that cut across sentences, or chunks too short to carry context).
- Stuck for >30 minutes? See `MP2_FAQ.md`, then ask in the cohort chat.

Constraints

- Use OpenAI (gpt-4o-mini + text-embedding-3-small) and Qdrant. No Anthropic, no Cohere, no Pinecone — same constraint as the capstone.
- No external frameworks (no LangChain, no LlamaIndex). Build the pipeline with the `openai` and `qdrant-client` libraries directly. The W9 deck showed what LlamaIndex would compress this to; MP2 deliberately stays from-scratch.
- Expected spend: \$0.05 - \$0.20 on OpenAI for the whole project, depending on how many test queries you run. Set a \$5 spend cap if you're nervous.

"You see, but you do not observe." — Sherlock Holmes, *A Scandal in Bohemia*.
Good luck. Have fun.